

Reinforcement Learning Basics

Shusen Wang

Stevens Institute of Technology

<http://wangshusen.github.io/>

A little bit probability theory...

一些概率论的方法

Random Variable

- **Random variable**: unknown; its values depend on outcomes of random events.

随机变量记作为X

可能发生的结果

随机事件

*Random
Variable*

*Possible
Values*

*Random
Events*

Probabilities

$$X = \begin{cases} 0 \\ 1 \end{cases}$$



$$\mathbb{P}(X = 0) = 0.5$$

$$\mathbb{P}(X = 1) = 0.5$$

Random Variable

- **Random variable**: unknown; its values depend on outcomes of random events.
- Uppercase letter X for a random variable.
- Lowercase letter x for an observed value. 通常用小写字母代表观测值，记为 x ，观测值是一个结果，没有随机性
- For example, I flipped a coin 4 times and observed:
 - $x_1 = 1$,
 - $x_2 = 1$,
 - $x_3 = 0$,
 - $x_4 = 1$.

Probability Density Function (PDF)

概率密度函数

- PDF provides a relative likelihood that the value of the random variable would equal that sample.

物理意义：意味着随机变量在某个确定的取值点附近的可能性

Probability Density Function (PDF)

- PDF provides a relative likelihood that the value of the random variable would equal that sample.

高斯分布 (正态分布)

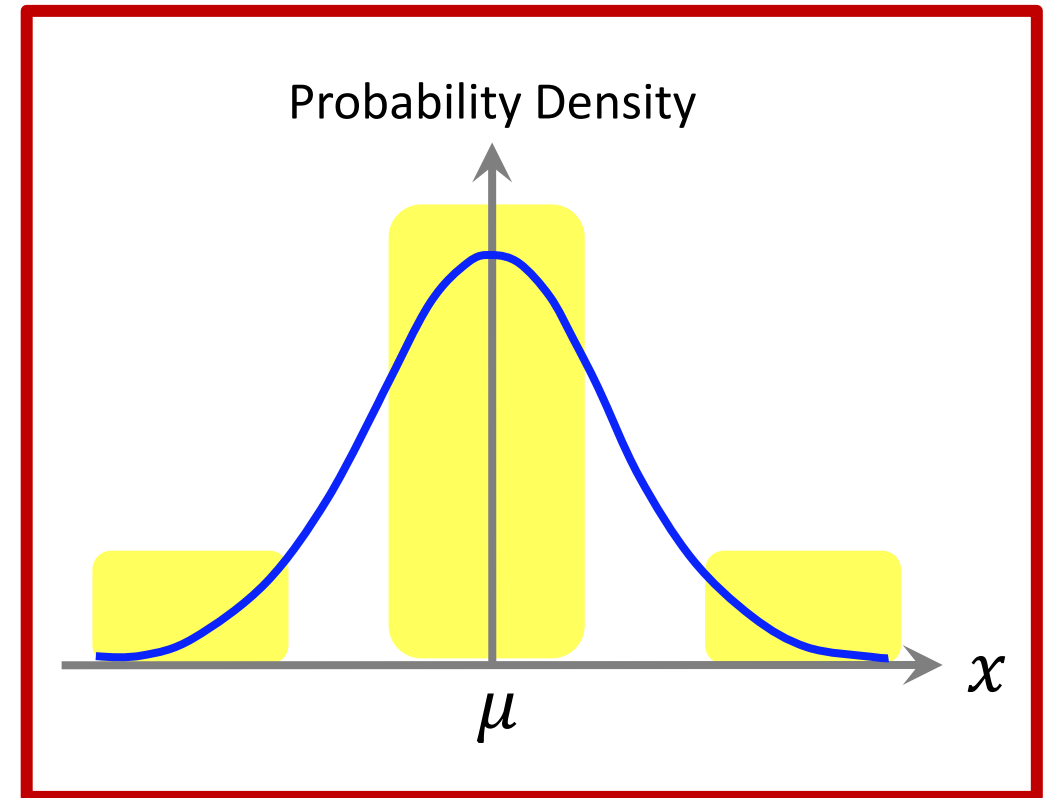
Example: Gaussian distribution

- It is a continuous distribution.
- PDF: 高斯分布的概率密度函数

σ 标准差, 方差越大, 数据越分散, 曲线越平缓; 方差越小, 数据越集中, 曲线越尖锐

连续型

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$



当x趋于均值μ的时候, 指数为0, 这部分最大为1, 此刻为最高点; x与均值μ相差越大, 这部分的值就呈指数级缩小, 曲线下降

Probability Mass Function (PMF)

- PMF is a function that gives the probability that a discrete random variable is exactly equal to some value.

Probability Mass Function (PMF)

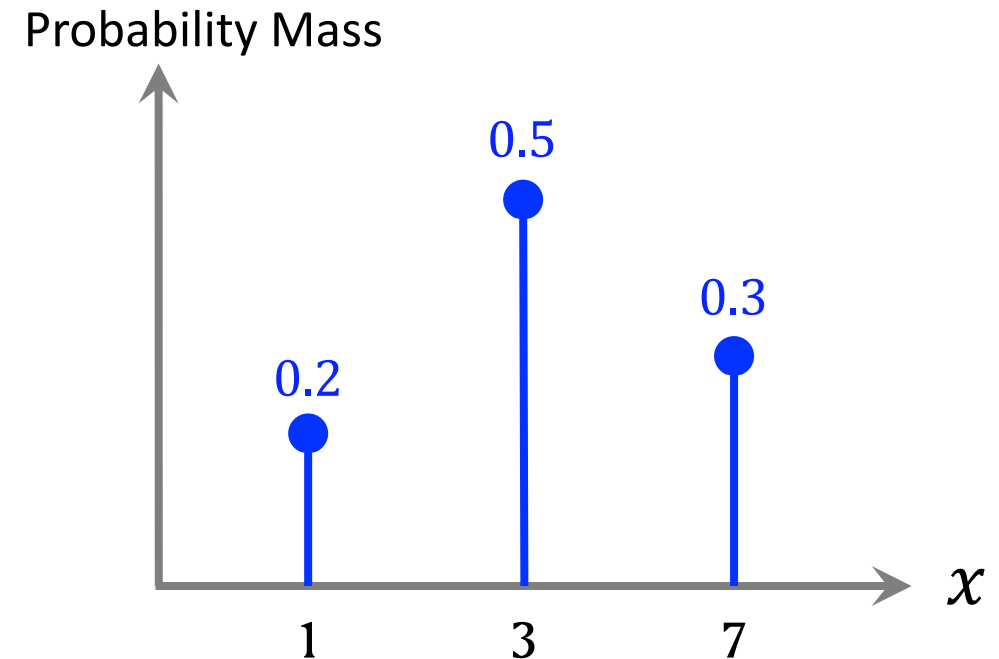
- PMF is a function that gives the probability that a discrete random variable is exactly equal to some value.

离散随机变量

Example

- Discrete random variable: $X \in \{1, 3, 7\}$.
- PDF:

$$\begin{aligned}p(1) &= 0.2, \\p(3) &= 0.5, \\p(7) &= 0.3.\end{aligned}$$



除此之外，在其他点的取值概率均为0

Probability Mass Function (PMF)

- PMF is a function that gives the probability that a discrete random variable is exactly equal to some value.

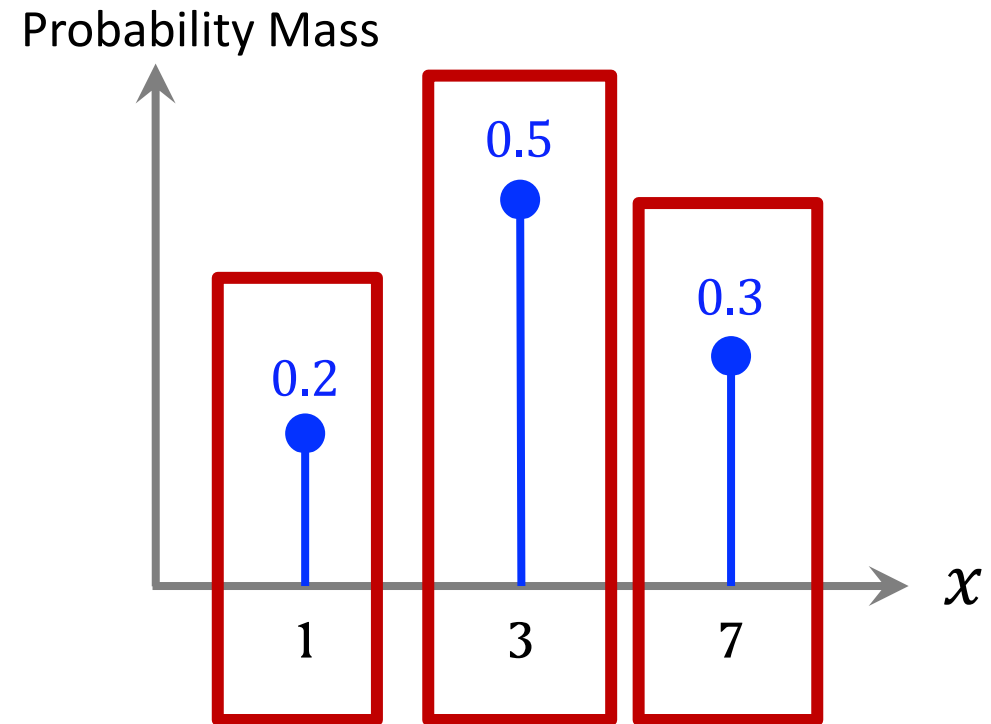
Example

- Discrete random variable: $X \in \{1, 3, 7\}$.
- PDF:

$$p(1) = 0.2,$$

$$p(3) = 0.5,$$

$$p(7) = 0.3.$$



Properties of PDF/PMF

- Random variable X is in the domain $\mathcal{X}$.

- For continuous distributions,

$$\int_{\mathcal{X}} p(x) dx = 1.$$

- For discrete distributions,

$$\sum_{x \in \mathcal{X}} p(x) = 1.$$

如果P是离散的概率分布，那么可以求每个点的概率和，概率和也为1

Expectation

期望

- Random variable X is in the domain $\mathcal{X}$.
- For continuous distributions, the expectation of $f(X)$ is:

$$\mathbb{E} [f(X)] = \int_{\mathcal{X}} p(x) \cdot f(x) dx.$$

连续

$p(x)$ 是概率密度函数

$f(x)$ 是原来的值

- For discrete distributions, the expectation of $f(X)$ is:

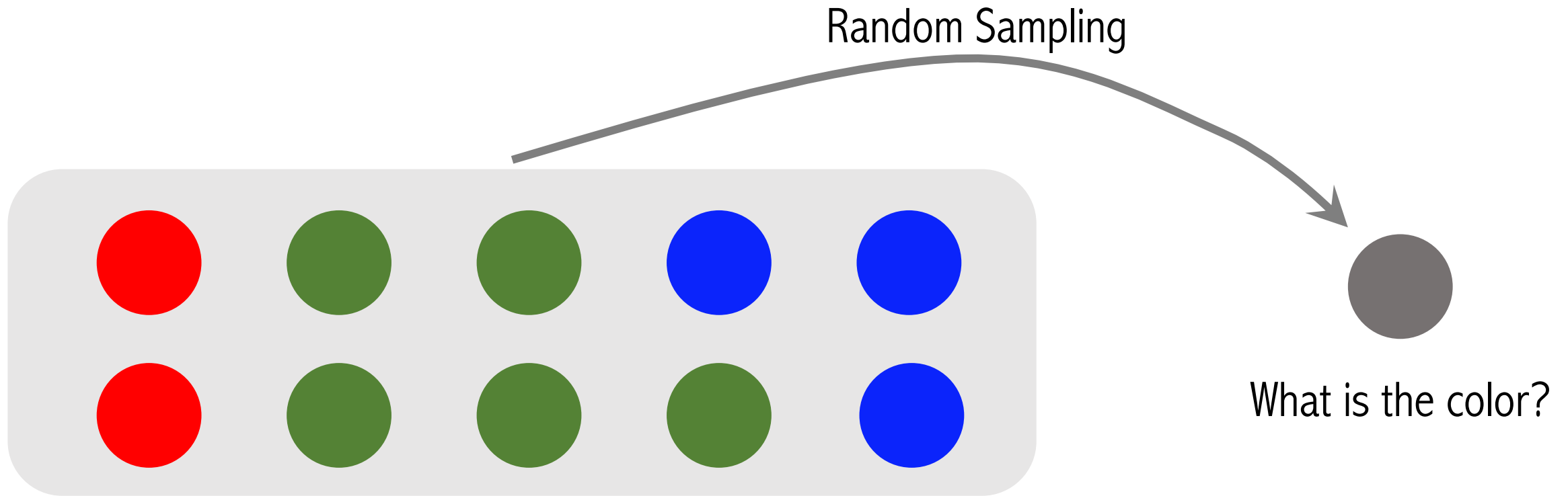
$$\mathbb{E} [f(X)] = \sum_{x \in \mathcal{X}} p(x) \cdot f(x).$$

离散

Random Sampling

随机抽样

- There are 10 balls in the bin: 2 are red, 5 are green, and 3 are blue.



Random Sampling

- Sample red ball w.p. 0.2, green ball w.p. 0.5, and blue ball w.p. 0.3.

Random Sampling

- Sample red ball w.p. 0.2, green ball w.p. 0.5, and blue ball w.p. 0.3.

```
from numpy.random import choice
```

大约20次记录是红色, 50次是绿色, 30次是蓝色

```
samples = choice(['R', 'G', 'B'], size=100, p=[0.2, 0.5, 0.3])
```

```
print(samples)
```

```
[ 'R'  'G'  'R'  'R'  'R'  'R'  'B'  'B'  'B'  'G'  'G'  'B'  'G'  'B'  'B'  'G'  'B'  'G'
  'B'  'B'  'G'  'B'  'G'  'B'  'B'  'G'  'B'  'B'  'G'  'B'  'G'  'G'  'G'  'G'  'G'  'B'
  'B'  'B'  'B'  'B'  'B'  'G'  'G'  'B'  'R'  'R'  'B'  'R'  'B'  'G'  'R'  'G'  'R'  'G'
  'R'  'R'  'B'  'G'  'G'  'G'  'B'  'R'  'G'  'B'  'G'  'R'  'G'  'G'  'G'  'B'  'B'  'R'
  'G'  'G'  'B'  'B'  'R'  'B'  'B'  'B'  'R'  'B'  'G'  'B'  'R'  'B'  'R'  'G'  'B'  'R'
  'B'  'B'  'G'  'G'  'G'  'R'  'R'  'B'  'R'  'G' ]
```

Terminologies

术语

Terminology: state and action

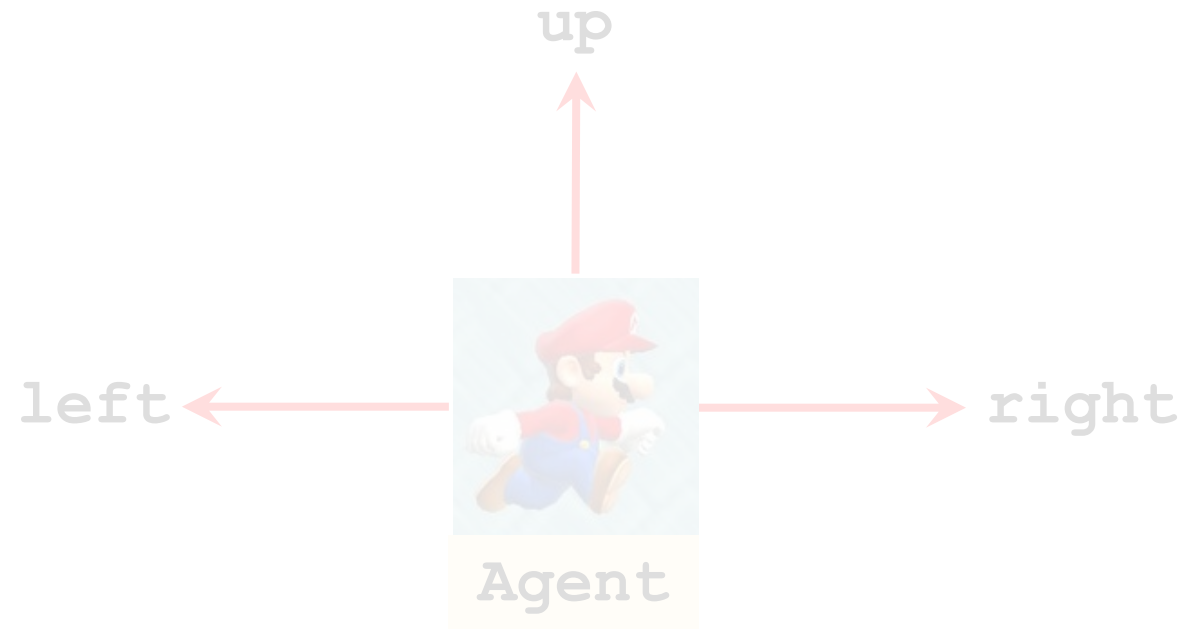
state s (this frame)

状态



知道Merry当前的状态

Action $a \in \{\text{left}, \text{right}, \text{up}\}$



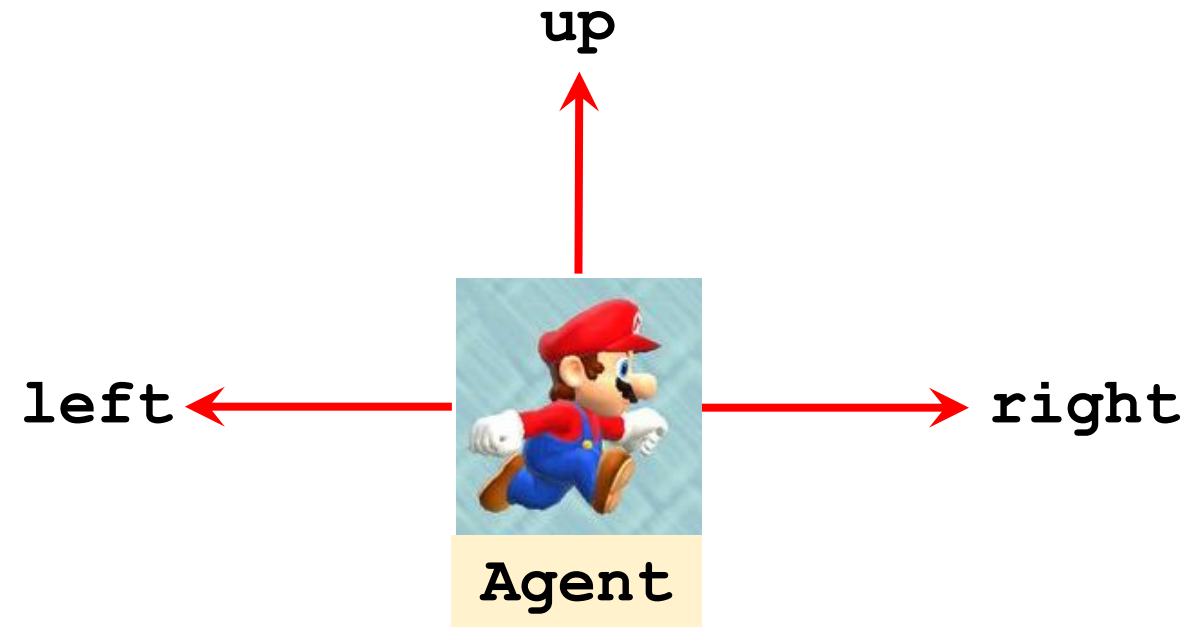
Terminology: state and action

state s (this frame)



Action $a \in \{\text{left}, \text{right}, \text{up}\}$

动作



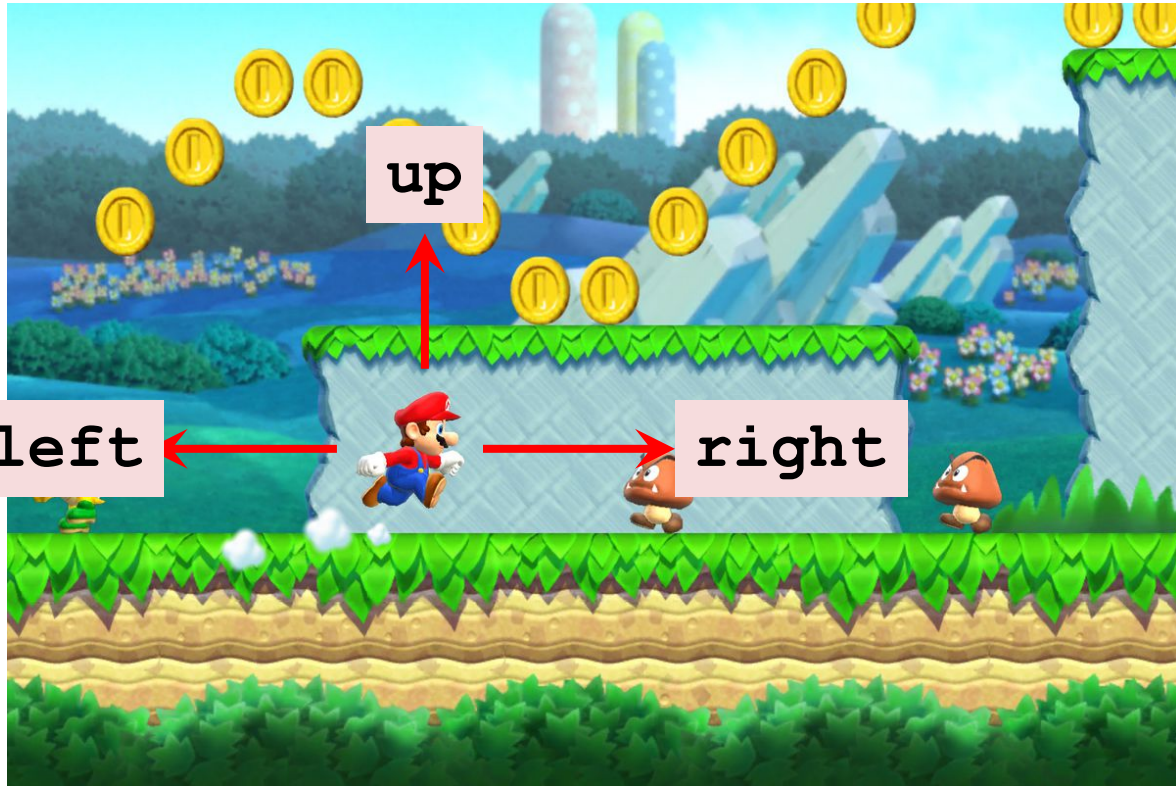
就可以操作Merry做一些动作了，向左向上向右都是动作

这里Merry被叫做Agent，如果是机器人，那么机器人就叫做Agent，总之，动作是谁做的谁就可以叫做Agent

Terminology: policy

Policy: 决策, 根据当前的State, 得出决策, 控制Agent的运动, 就叫做Policy

向上跳可以吃到金币



policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:

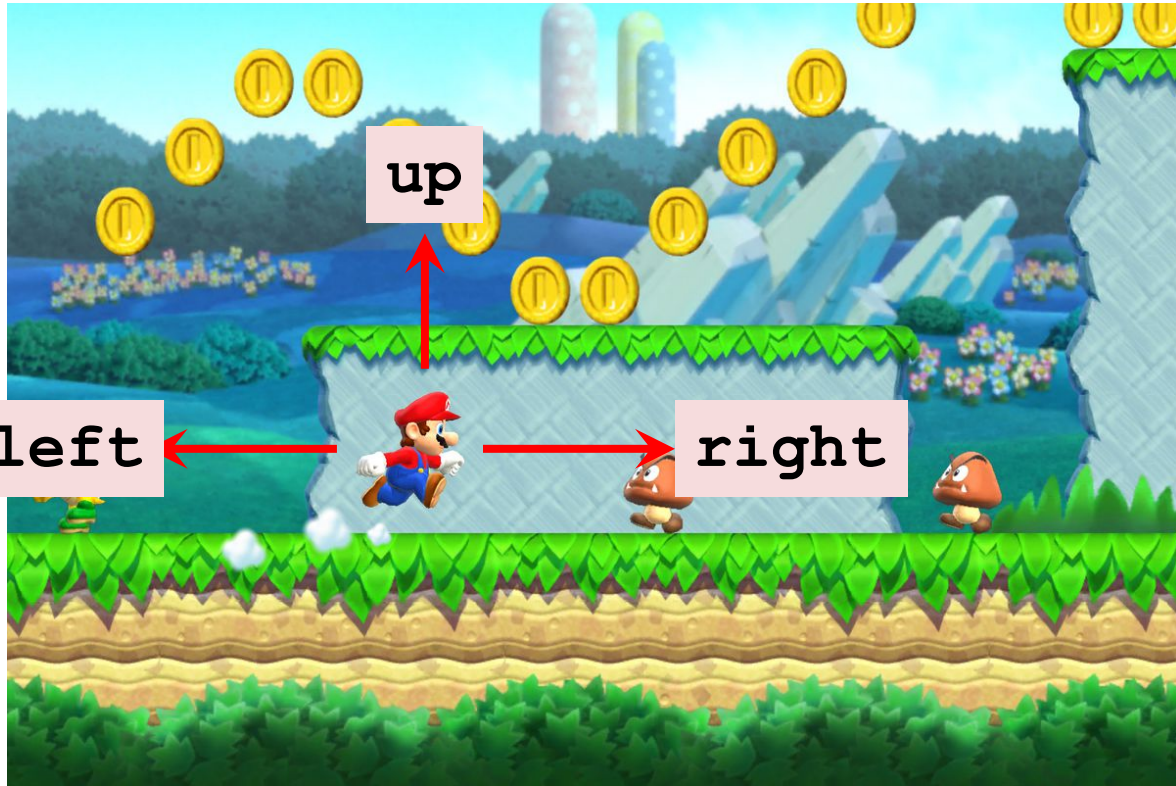
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$

Terminology: policy

policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:

$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$



Terminology: policy

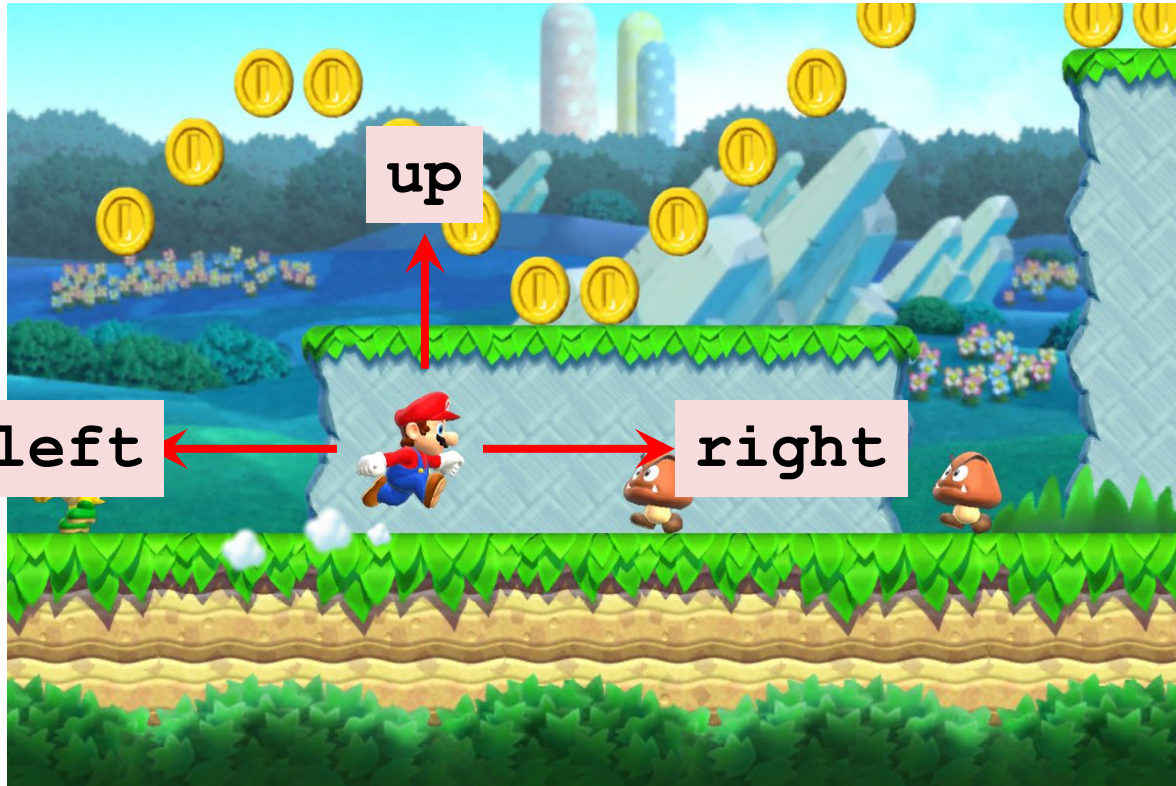
policy π

数学上, Policy π 是概率密度函数

- Policy function $\pi: (s, a) \mapsto [0,1]$:

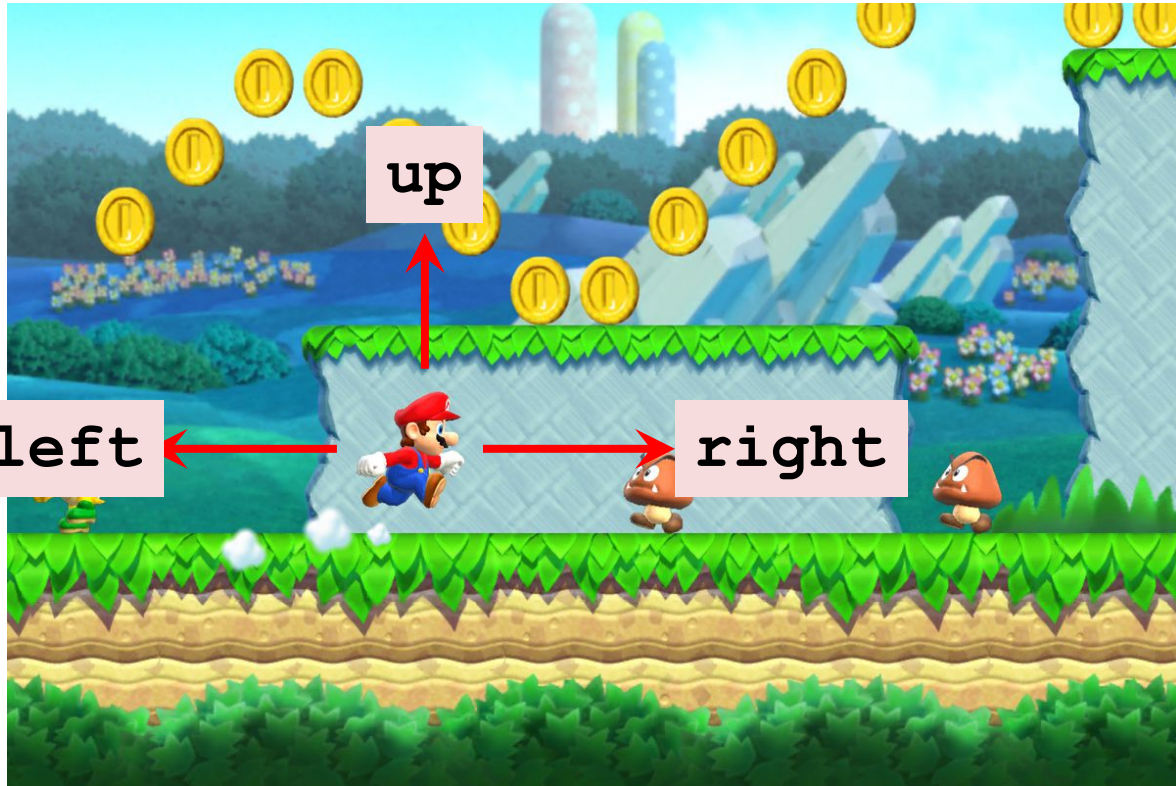
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$

π (在s的条件下a的概率) = 给出状态S, 做出动作A的概率密度



Terminology: policy

policy π



- $\pi(a | s)$ is the probability of taking action $A = a$ given state s , e.g.,

➔ • $\pi(\text{left} | s) = 0.2,$

➔ • $\pi(\text{right} | s) = 0.1,$

➔ • $\pi(\text{up} | s) = 0.7.$

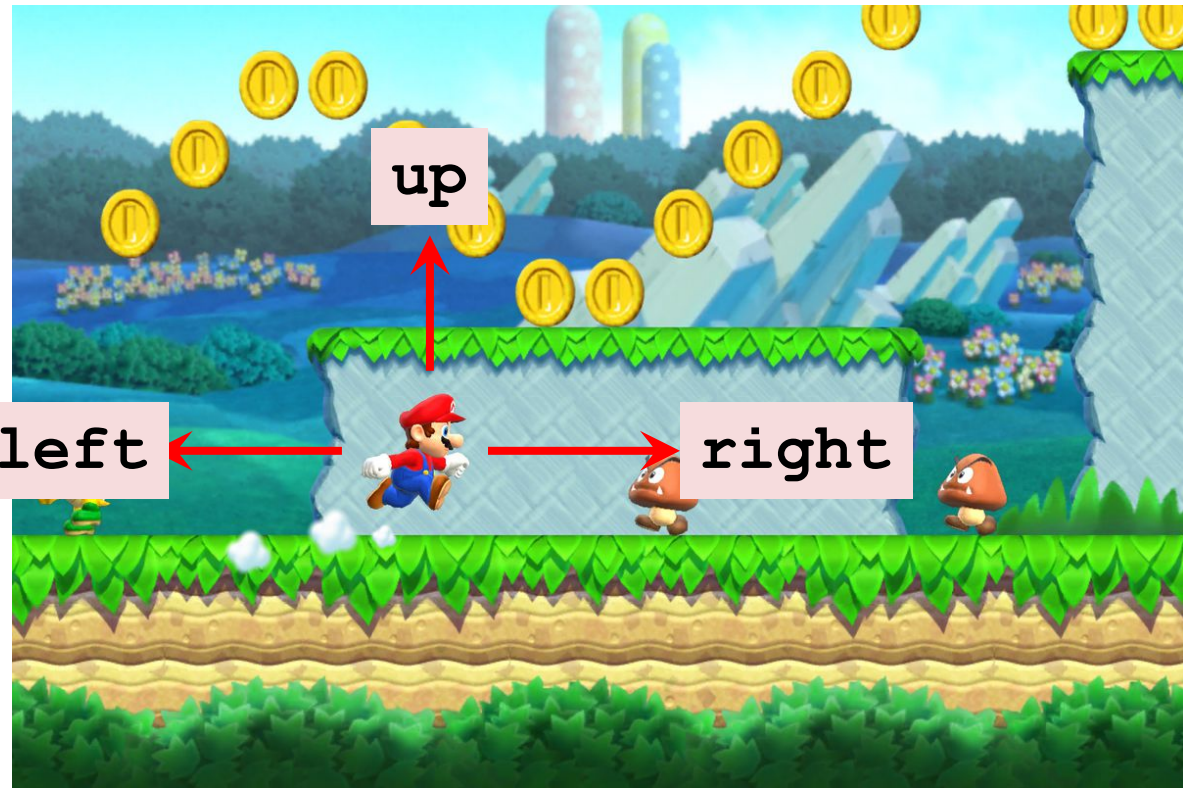
如上所示

Merry的动作可以是随机或者确定，但是最好还是随机

比如猜拳，如果Action是固定的，则可以找到规律，所以最好还是随机

Terminology: policy

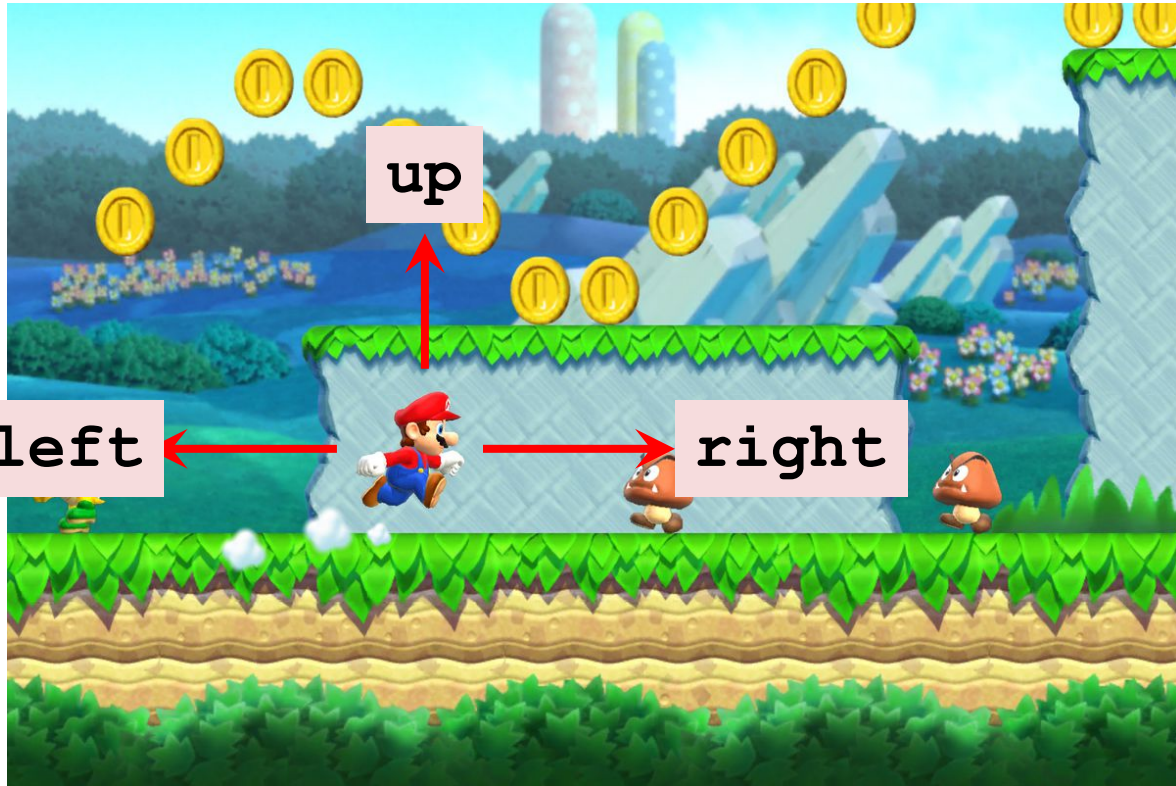
policy π



- $\pi(a | s)$ is the probability of taking action $A = a$ given state s , e.g.,
 - $\pi(\text{left} | s) = 0.2$,
 - $\pi(\text{right} | s) = 0.1$,
 - $\pi(\text{up} | s) = 0.7$.
- Upon observing state $S = s$, the agent's action A can be random.

Terminology: policy

Random or deterministic policy?



Terminology: reward

reward R

- Collect a coin: $R = +1$

吃到进步的奖励 + 1



Terminology: reward

reward R

- Collect a coin: $R = +1$
- Win the game: $R = +10000$

打赢游戏的奖励分数很高，这样才能激励打赢游戏而不是为了吃金币



Terminology: reward

reward R

- Collect a coin: $R = +1$
- Win the game: $R = +10000$
- Touch a Goomba: $R = -10000$ (game over).

如果碰到敌人，Merry就死了，GameOver所以是-10000



Terminology: reward

reward R



- Collect a coin: $R = +1$
- Win the game: $R = +10000$
- Touch a Goomba: $R = -10000$ (game over).
- Nothing happens: $R = 0$

当前没有发生则奖励为0

Terminology: state transition

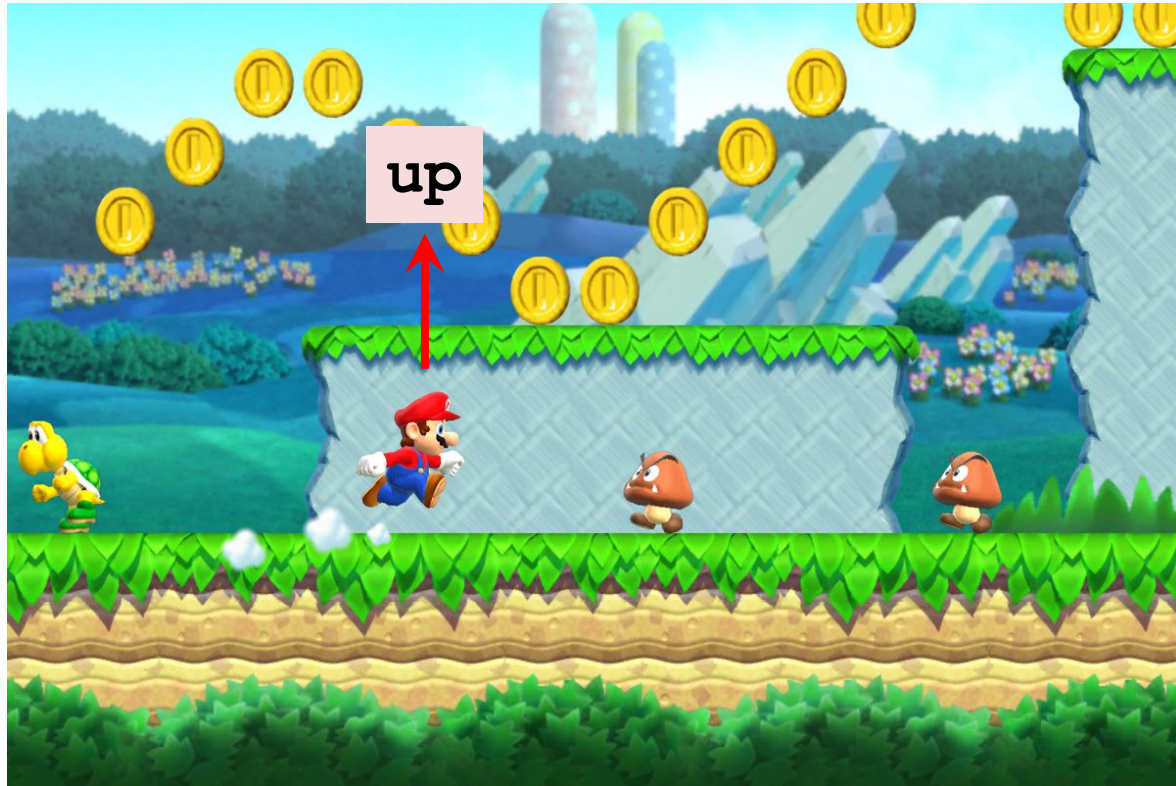
状态转移

state transition



Terminology: state transition

state transition



- E.g., “up” action leads to a new state.

Terminology: state transition

state transition

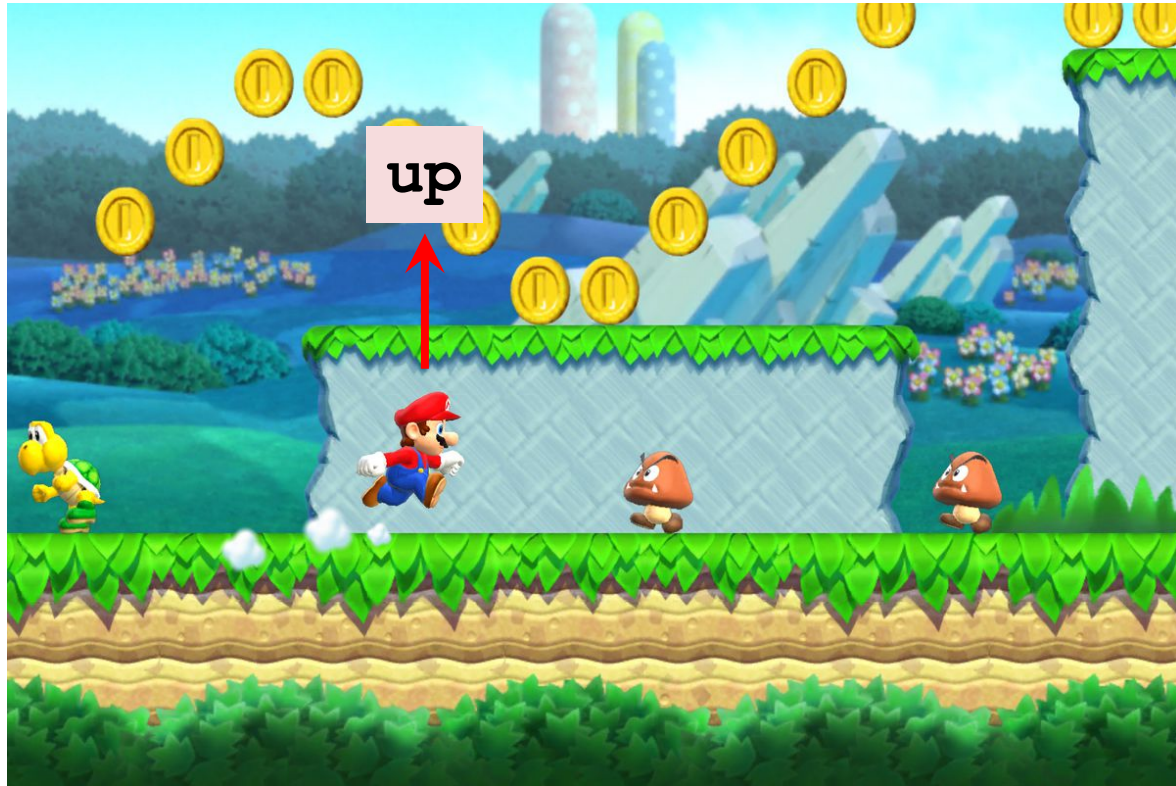


- E.g., “up” action leads to a new state.

状态转移可以是确定的也可以是随机的，但通常是随机的

- State transition can be random.
- Randomness is from the environment.

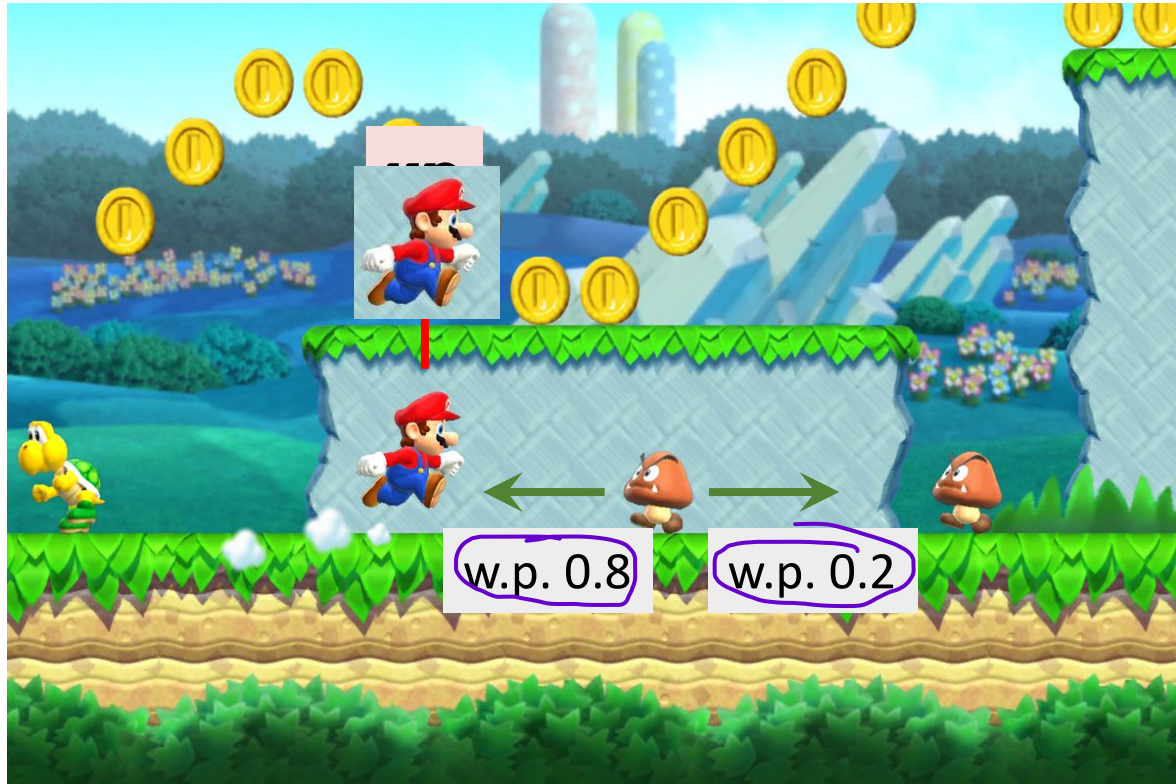
状态转移的随机性通常是通过环境得到的



如图，Merry向上跳，Merry的状态就在上面了，此时是确定的，但怪物可以向左也可以向右，下一步的状态是随机的

Terminology: state transition

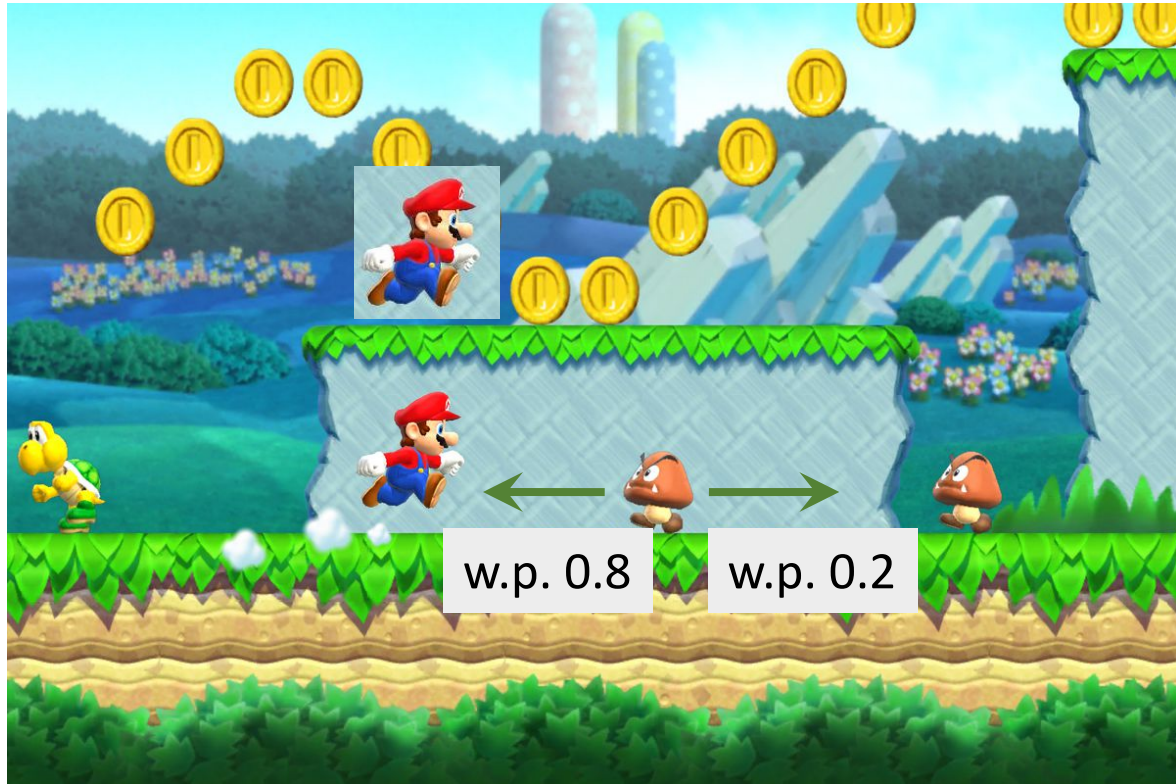
state transition



- E.g., “up” action leads to a new state.
- State transition can be random.
- Randomness is from the environment.

Terminology: state transition

state transition



- E.g., “up” action leads to a new state.
- State transition can be random.
- Randomness is from the environment.
- $p(s'|s, a) = \mathbb{P}(S' = s' | S = s, A = a)$.

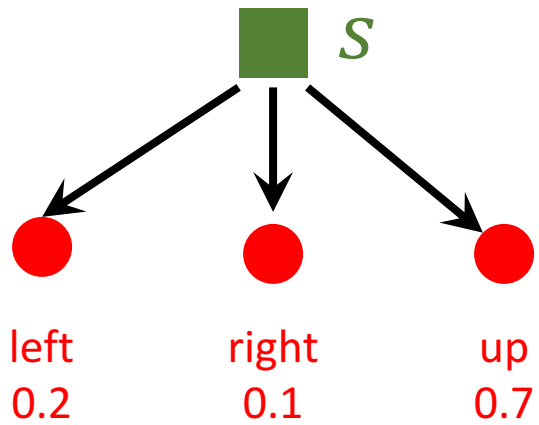
意思是如果观测到当前的状态 $S=s, A=a$, 输出状态变成 s' 的概率

状态转移可以通过P函数来表示, 这是一个条件概率密度函数

知道怪物是往左和往右, 但是不确定最后的结果是往左和往右, 所以结果只有环境自己知道, 玩家是不知道的

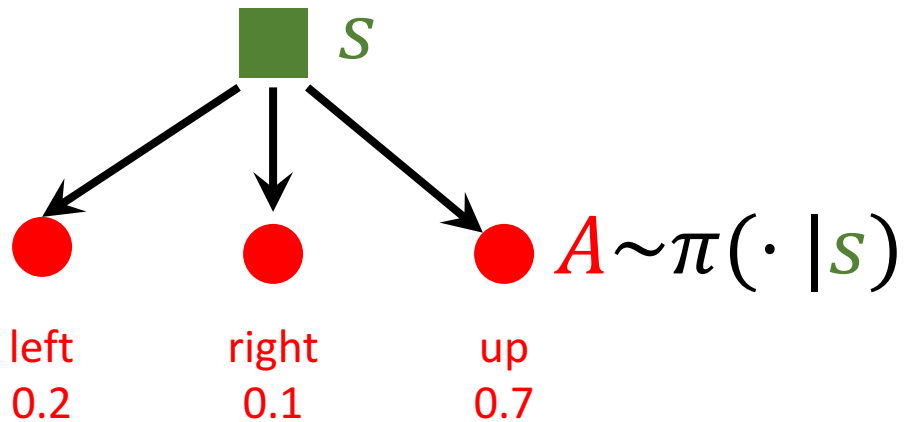
Two Sources of Randomness

Randomness in Actions



Randomness in Actions

强化学习中的随机性

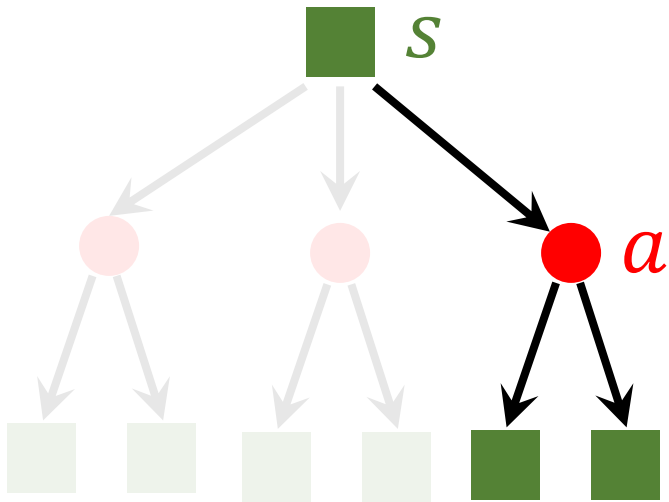


Given state s , the action can be random, e.g., .

- $\pi(\text{"left"}|s) = 0.2,$
- $\pi(\text{"right"}|s) = 0.1,$
- $\pi(\text{"up"}|s) = 0.7.$

①Agent可以做任何动作

Randomness in States



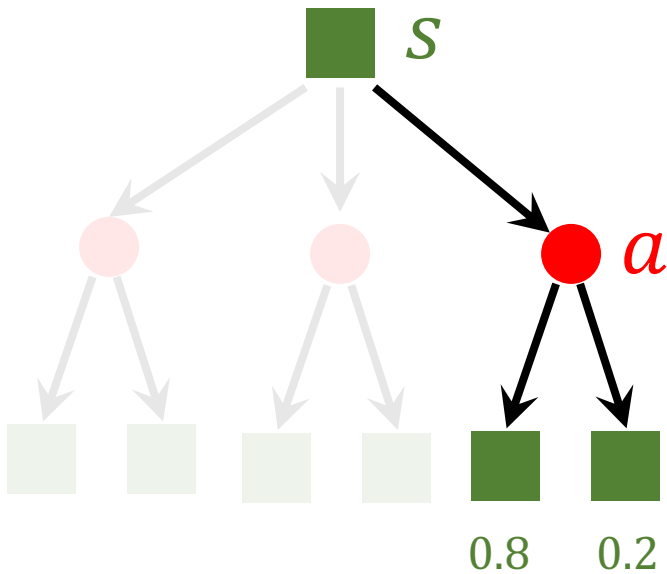
- State transition can be random.
- The environment generates the new state s' by

$$\underline{s' \sim p(\cdot | s, a) .}$$

Randomness in States

- State transition can be random.
- The environment generates the new state S' by

$$S' \sim p(\cdot | s, a) .$$



②状态转移，假如做出了向上跳的动作，环境就要生成下一个状态 S' ，这个新状态有随机性，环境用状态转移函数 p 计算出概率，然后用概率来随机抽样得到下一个状态 S'

Two Sources of Randomness

- The randomness in **action** is from the policy function:

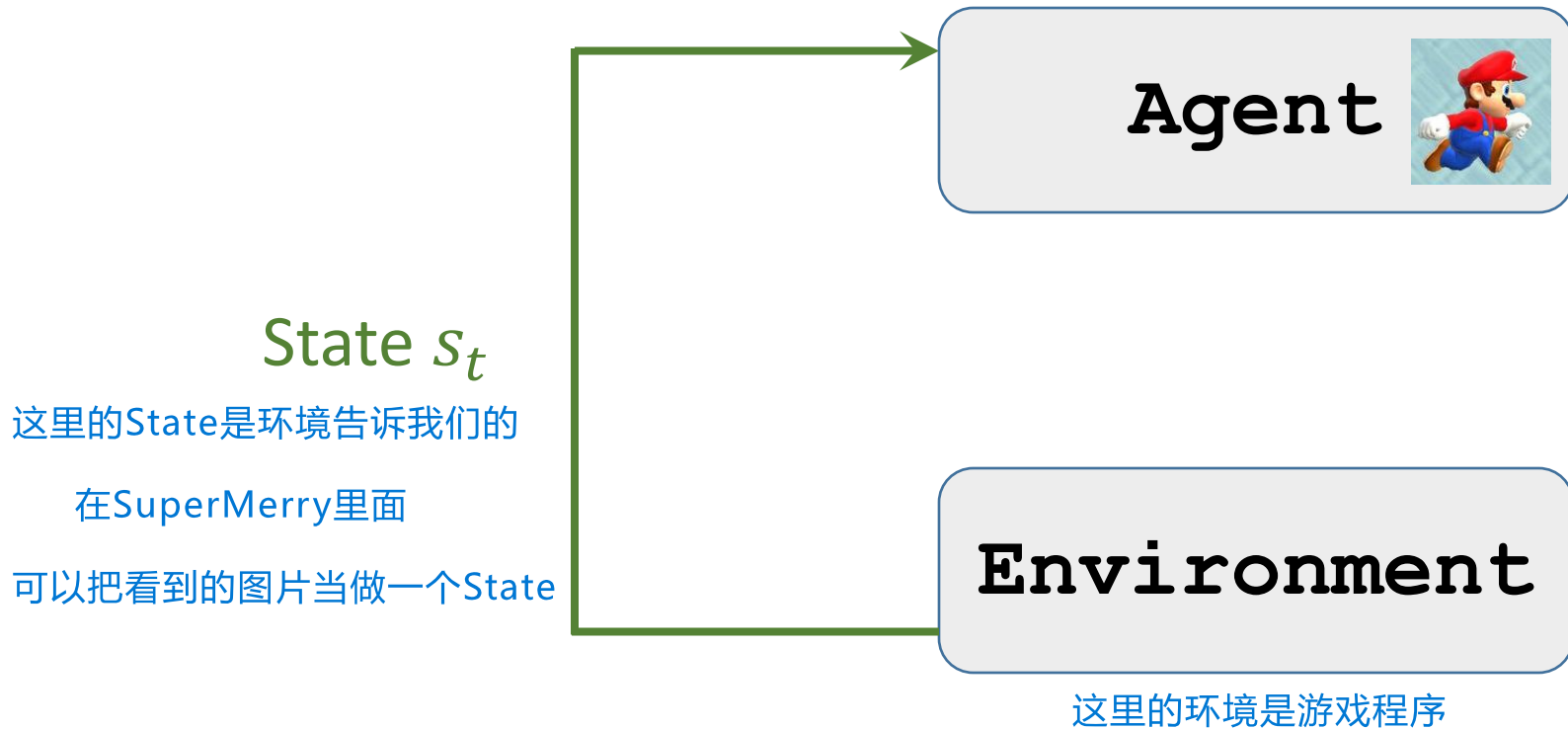
$$A \sim \pi(\cdot \mid s) .$$

- The randomness in **state** is from the state-transition function:

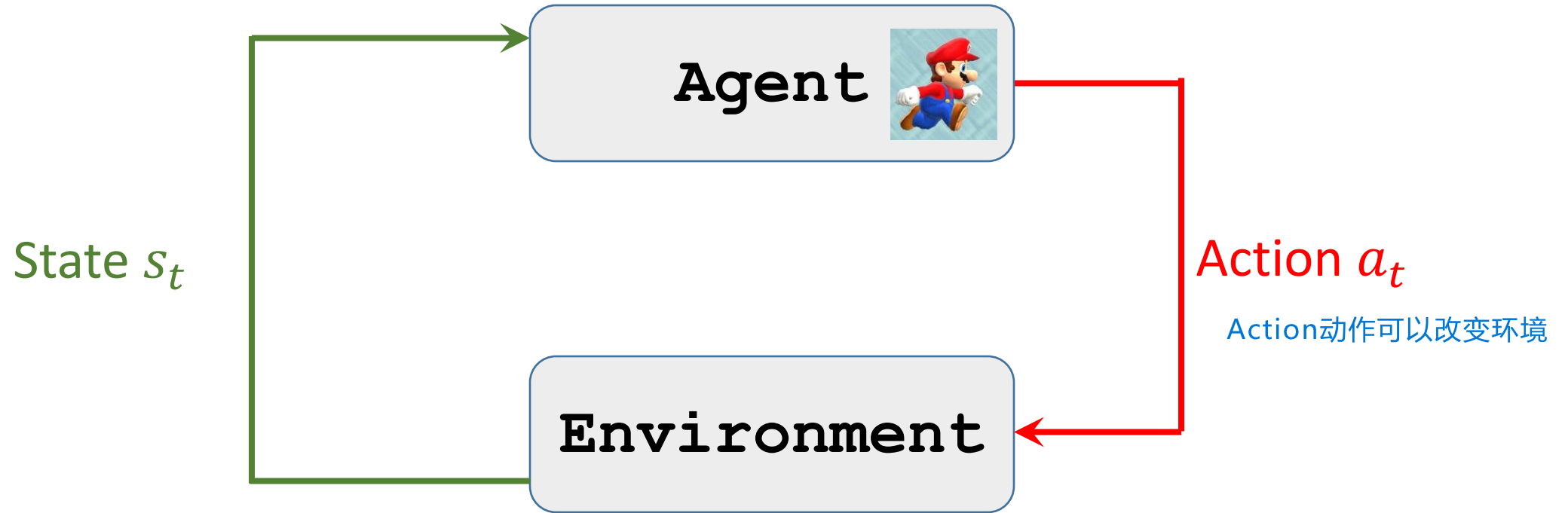
$$S' \sim p(\cdot \mid s, a) .$$

Agent-Environment Interaction

Agent-Environment Interaction



Agent-Environment Interaction



Agent-Environment Interaction



Play game using AI

怎么样让AI自动打赢游戏呢

- Observe state s_t , select action $a_t \sim \pi(\cdot | s_t)$, and execute a_t .

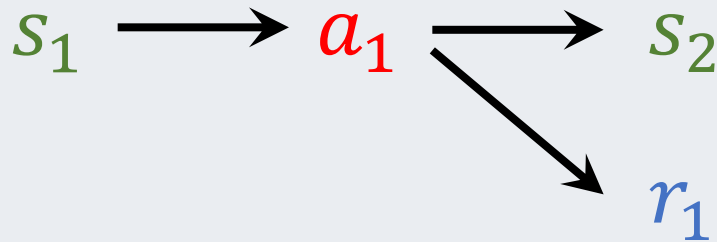
观察当前的状态State, 根据概率函数 π 选择Action并执行动作Action

- The environment gives new state s_{t+1} and reward r_t .

得到新的状态State+1和奖励Return

Play game using AI

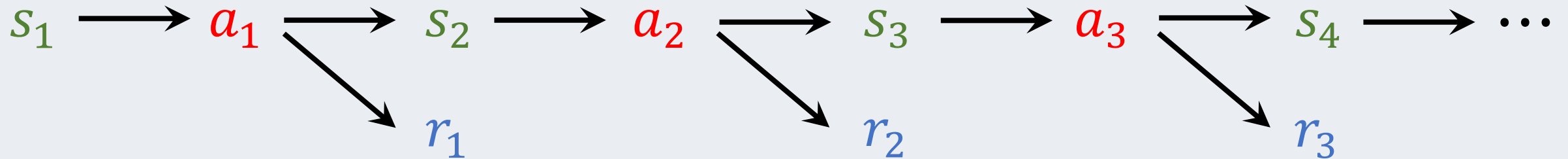
- Observe state s_t , select action $a_t \sim \pi(\cdot | s_t)$, and execute a_t .
- The environment gives new state s_{t+1} and reward r_t .



Play game using AI

- Observe state s_t , select action $a_t \sim \pi(\cdot | s_t)$, and execute a_t .
- The environment gives new state s_{t+1} and reward r_t .

持续这样下去直到结束游戏

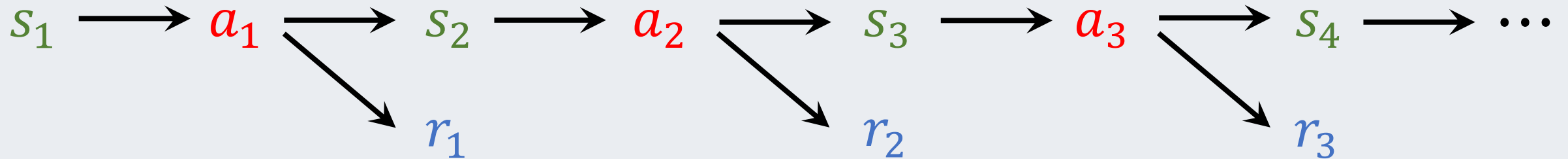


Play game using AI

- (state, action, reward) trajectory: 然后我们就得到了游戏的轨迹trajectory

$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_n, a_n, r_n$

- One episode is from the the beginning to the end (Mario wins or dies).



Play game using AI

- (state, action, reward) trajectory:

$$s_1, a_1, r_1, \quad s_2, a_2, r_2, \quad \cdots, \quad s_n, a_n, r_n.$$

- One episode is from the the beginning to the end (Mario wins or dies).
- What is a good policy?
- A good policy leads to big cumulative reward: $\sum_{t=1}^n \gamma^{t-1} \cdot r_t$.
- Use the rewards to guide the learning of policy.

Rewards and Returns

奖励和汇报

Return

回报

翻译为：未来的累计奖励

Definition: Return (aka cumulative future reward).

$$\bullet \quad \underline{U_t} = \underline{R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots}$$

每次的奖励的总和为回报

Return

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Question: At time t , are R_t and R_{t+1} equally important?

- Which of the followings do you prefer? 奖励 R_t 和 R_{t+1} 同样重要吗
 - I give you \$100 right now. 现在给100块钱
 - I will give you \$100 one year later. 一年后得到100块钱

Return

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Question: At time t , are R_t and R_{t+1} equally important?

- Which of the followings do you prefer? 换一个问题
 - I give you \$80 right now. 现在得到80
 - I will give you \$100 one year later. 一年后给你100

Return

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Question: At time t , are R_t and R_{t+1} equally important?

- Which of the followings do you prefer?
 - I give you \$80 right now.
 - I will give you \$100 one year later.

未来奖励100没有现在的奖励100好，未来的100可能在现在就值80

- Future reward is less valuable than present reward.
- R_{t+1} should be given less weight than R_t .

所以要给未来的奖励打一个折扣，比如8折：100 * 0.8 = 现在的80

所以未来奖励 R_{t+1} 的权重要比 R_t 小

Discounted Returns

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

折扣回报

Definition: Discounted return (aka cumulative discounted future reward).

- γ : discount factor (tuning hyper-parameter). 折扣率 超参数, 需要自己选择和调整 γ 介于0-1之间

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

如果一样重要则 $\gamma=1$

如果未来的奖励不重要则 γ 就越小

Discounted Returns

Definition: Discounted return (at time t).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

Randomness in Returns

Definition: Discounted return (at time t).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

At the end of the game, we observe u_t .

- We observe all the rewards, $r_t, r_{t+1}, r_{t+2}, \dots, r_n.$
- We thereby know the discounted return:

$$u_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{n-t} r_n.$$

Randomness in Returns

Return的随机性

Definition: Discounted return (at time t).

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

At time t , the rewards, $R_t, \dots, R_n$, are **random**, so the return U_t is **random**.

假如游戏已经结束了，则奖励已经知道了，是数值，用小写字母r来表示；

如果在t时刻游戏还没有结束，这些奖励都是随机值，用大写字母R来表示，Return（U）依赖于奖励R，所以U也用大写字母表示

Randomness in Returns

Definition: Discounted return (at time t).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

At time t , the rewards, $R_t, \dots, R_n$, are **random**, so the return U_t is **random**.

- Reward R_i depends on S_i and A_i . R_i 依赖于 S_i 和 A_i
- States can be random: $S_i \sim p(\cdot | s_{i-1}, a_{i-1}).$ States是随机的
- Actions can be random: $A_i \sim \pi(\cdot | s_i).$ Actions也是随机的
- If either S_i or A_i is random, then R_i is random.

因为States和Actions都是随机的，所以Reward也是随机的

Randomness in Returns

Definition: Discounted return (at time t).

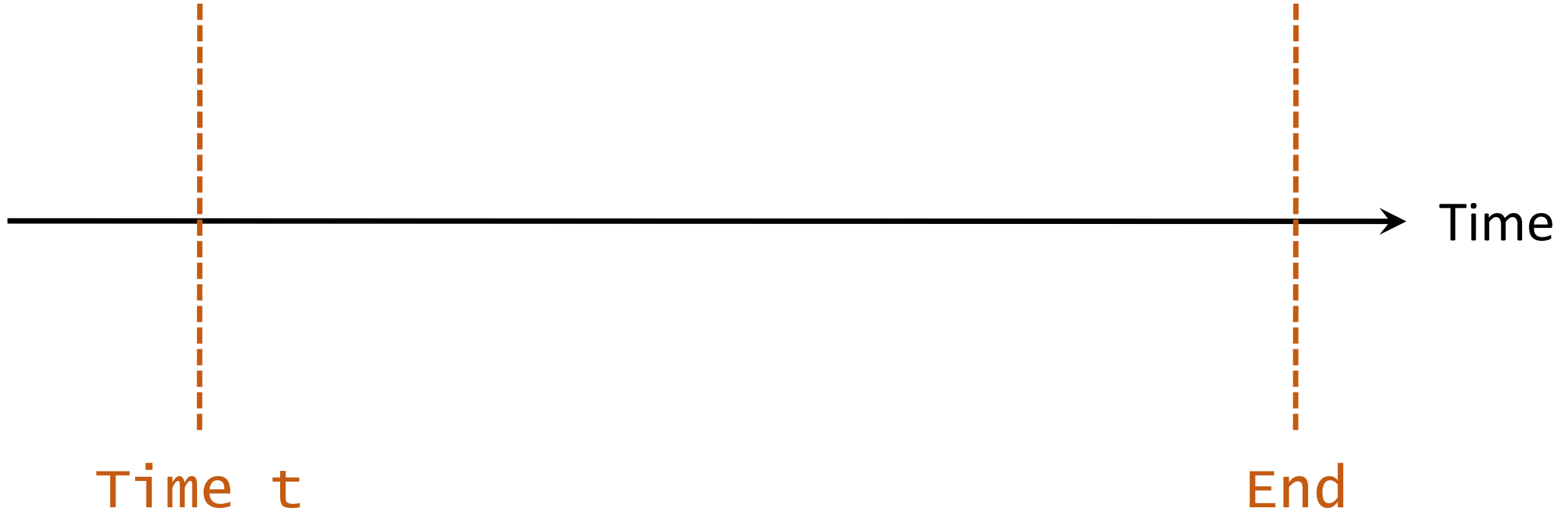
- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

At time t , the rewards, $R_t, \dots, R_n$, are **random**, so the return U_t is **random**.

- Reward R_i depends on S_i and A_i .
- U_t depends on $R_t, R_{t+1}, \dots, R_n$. 而回报Return依赖于Rewards
- $\Rightarrow U_t$ depends on $S_t, A_t, S_{t+1}, A_{t+1}, \dots, S_n, A_n$.

所以Return回报与States和Actions等等有关

Randomness in Returns



Randomness in Returns



R_t

R_{t+1}

R_{t+2}

...

R_n

U_t is a random variable

在Time = t时刻游戏还没有结束

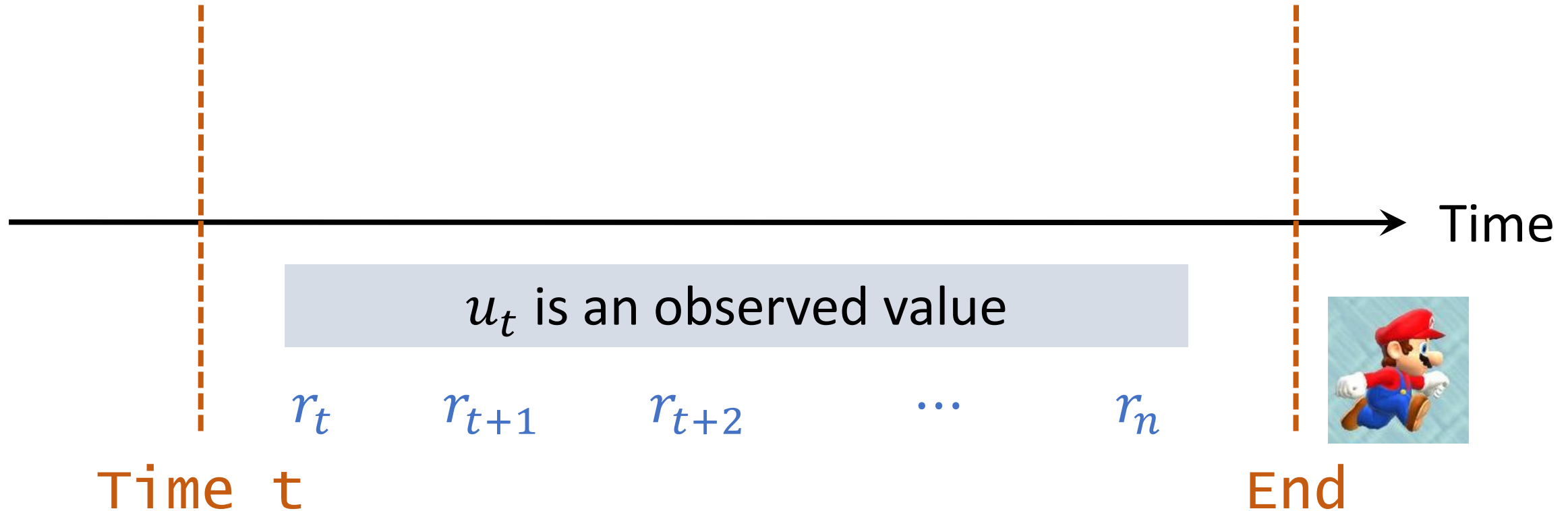
此时, Reward用大写表示, U_t 也是随机变量

Time t

End

Time

Randomness in Returns



End, 游戏结束了, Rewards用小写表示

此时 U_t 是一个数值

Value Functions

价值函数

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

在T时刻，并不知道 U_t 是什么

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

Definition: Action-value function.

- $Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$



U_t depends on states $s_t, s_{t+1}, \cdots, s_n$ and actions $a_t, a_{t+1}, \cdots, a_n$.

由于 U_t 是随机变量，依赖于未来所有的动作Actions以及未来所有状态States

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$

Regard s_t and a_t as observed values.

Regard $s_{t+1}, \cdots, s_n$ and $a_{t+1}, \cdots, a_n$ as random variables.

这些参数作为随机变量被积分掉了

由于 U_t 是随机变量，在 T 时刻我们并不知道 U_t 是什么，怎样评估 T 时刻的形势

Action-Value Function $Q_\pi(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_\pi(s_t, a_t) = \mathbb{E}[U_t \mid S_t = s_t, A_t = a_t].$$

对 U_t 求期望，把随机性用积分积掉

Regard $s_{t+1}, \cdots, s_n$ and $A_{t+1}, \cdots, A_n$ as random variables.

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E}[U_t \mid S_t = s_t, A_t = a_t].$$

将 U_t 当做未来所有的动作 A 和状态 S 的函数
动作 A 和状态 S 都有随机性

状态函数的概率密度函数是状态转移函数 p

$$\begin{aligned} \bullet S_{t+1} &\sim p(\cdot \mid s_t, a_t), \\ &\vdots \\ \bullet S_n &\sim p(\cdot \mid s_{n-1}, a_{n-1}). \end{aligned}$$

动作 A 的概率密度函数是Policy π

$$\begin{aligned} \bullet A_{t+1} &\sim \pi(\cdot \mid s_{t+1}), \\ &\vdots \\ \bullet A_n &\sim \pi(\cdot \mid s_n). \end{aligned}$$

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

求期望得到的Q函数被称为Action Values Function, 动作价值函数

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$



期望就是对未来的动作Actions和状态States求的, 将随机变量都用积分给积掉

除了 S_t 和 A_t 其余随机变量都被积掉了

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

Definition: Action-value function.

- $Q_{\pi}(\underline{s_t}, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$

Action-Value Function $Q_{\pi}(s, a)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$

Q π 和当前的状态State和Action (Time=t) 有关

- $Q_{\pi}(s_t, a_t)$ depends on s_t, a_t, π , and p .
- $Q_{\pi}(s_t, a_t)$ is dependent of $s_{t+1}, \cdots, s_n$ and $a_{t+1}, \cdots, a_n$.

当前的State和Action被当作观测到的数值来对待，而随机变量被积分掉了

State-Value Function $V_{\pi}(s)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t]. \quad Q^*(s_t, a_t) = \max_{\pi} Q_{\pi}$$

Definition: State-value function.

观测到一个状态如果向上跳分数更高，则会向上跳

状态价值函数

$$\bullet \underline{V_{\pi}(s_t)} = \mathbb{E}_{\underline{A}} [\underline{Q_{\pi}(s_t, A)}]$$

VP是动作价值函数QP的期望

State-Value Function $V_{\pi}(s)$

Definition: Discounted return.

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$

Definition: Action-value function.

- $Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$

Definition: State-value function.

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)]$

关于随机变量A求

Taken w.r.t. the action $A \sim \pi(\cdot \mid s_t).$

State-Value Function $V_{\pi}(s)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$

Definition: State-value function.

$$\bullet V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a).$$

动作为离散的，比如上下左右
(Actions are discrete.)

Taken w.r.t. the action $A \sim \pi(\cdot | s_t)$.

State-Value Function $V_{\pi}(s)$

Definition: Discounted return.

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n.$$

Definition: Action-value function.

$$\bullet Q_{\pi}(s_t, a_t) = \mathbb{E} [U_t \mid S_t = s_t, A_t = a_t].$$

Definition: State-value function.

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a).$ (Actions are discrete.)
- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \int \pi(a|s_t) \cdot Q_{\pi}(s_t, a) da.$ (Actions are continuous.)

动作里离散的，比如驾车方向盘的角度，在一个区间内变化

Understanding the Value Functions

对U（未来的回报，未知）在状态S和动作A的条件下计算条件概率

- **Action**-value function: $Q_{\pi}(s, a) = \mathbb{E} [U_t | S_t = s, A_t = a]$.
- Given policy π , $Q_{\pi}(s, a)$ evaluates how good it is for an agent to pick action a while being in state s .

Q π 说明处在状态s做出动作a是否明智，可以给动作a打分

Understanding the Value Functions

- **Action**-value function: $Q_{\pi}(s, a) = \mathbb{E} [U_t | S_t = s, A_t = a]$.
- Given policy π , $Q_{\pi}(s, a)$ evaluates how good it is for an agent to pick action a while being in state s .

V把Q中的A通过积分给去掉，这样变量只剩下状态S

- **State**-value function: $V_{\pi}(s) = \mathbb{E}_A [Q_{\pi}(s, A)]$ V和状态State有关，和动作Action无关
- For fixed policy π , $V_{\pi}(s)$ evaluates how good the situation is in state s .
- $\mathbb{E}_S [V_{\pi}(S)]$ evaluates how good the policy π is. 可以通过V判断当前状态是好是坏

V还可以判断Policy π 函数的好坏，Policy越好，则V的平均值越大

Evaluating Reinforcement Learning

1. 学习Policy函数 π 策略学习, 加入有Policy函数 π 可以通过这个函数控制动作

2. 学习 Q^* , 价值学习, 假如有了 Q^* , Q^* 告诉我们, 假如我们处在这个状态, 这个状态是好是坏, 则可以控制动作

$$a_t = \operatorname{argmax}_a Q^*(s_t, a)$$

两种方法都可行, 强化学习就是学习Policy或者 Q^*

OpenAI Gym

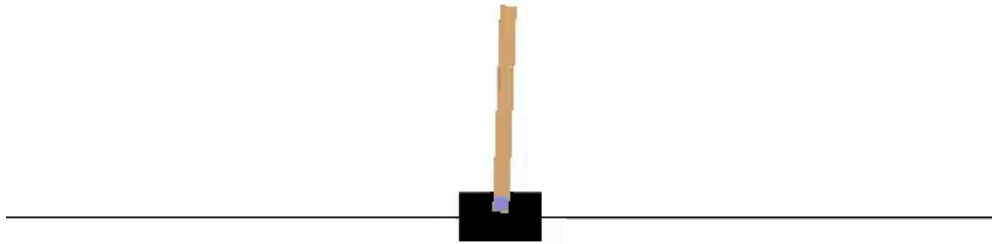
- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

OpenAI Gym

- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

经典控制问题

Classical control problems



给小车一个左右的力，让杆子恰好能竖起来

Cart Pole



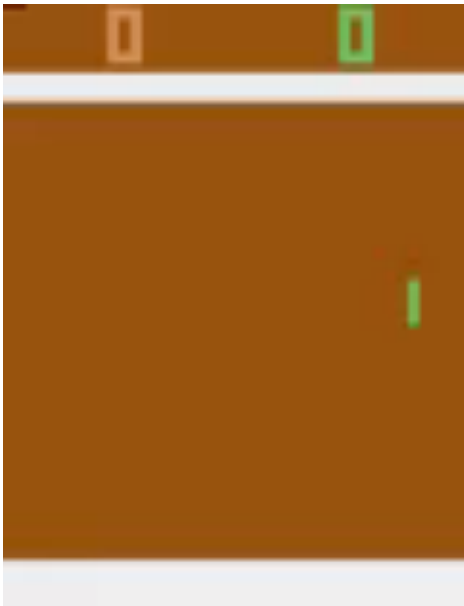
给钟摆一个力，让钟摆也能够刚好竖起来

Pendulum

OpenAI Gym

- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

Atari Games



Pong



Space Invader



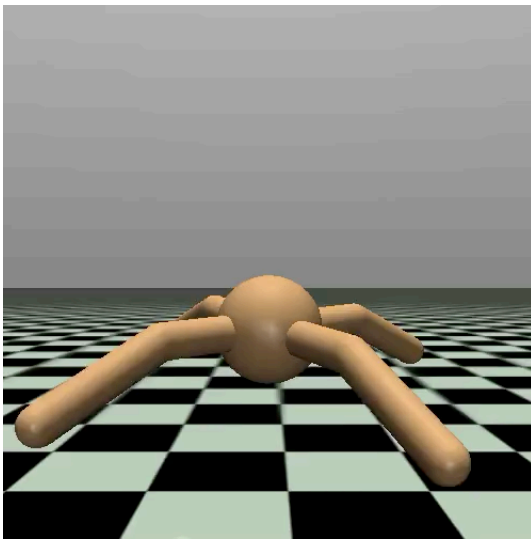
Breakout

OpenAI Gym

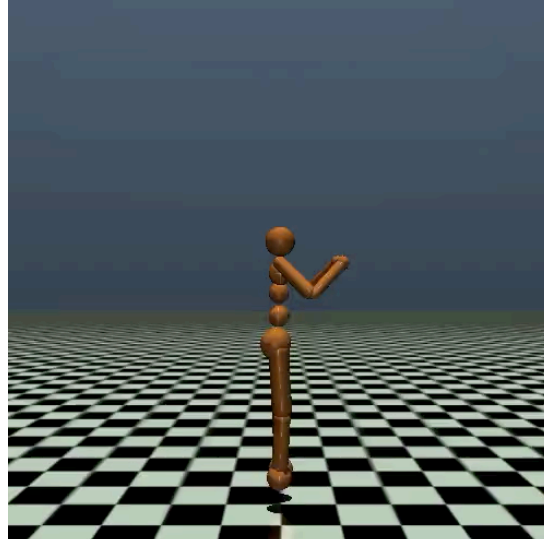
- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

模拟问题

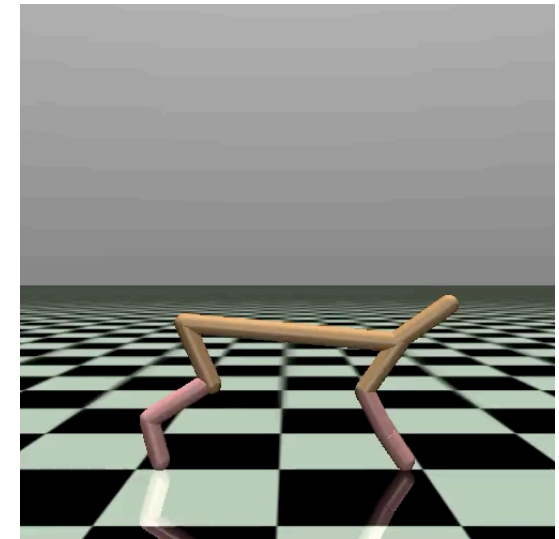
MuJoCo (Continuous control tasks.)



Ant



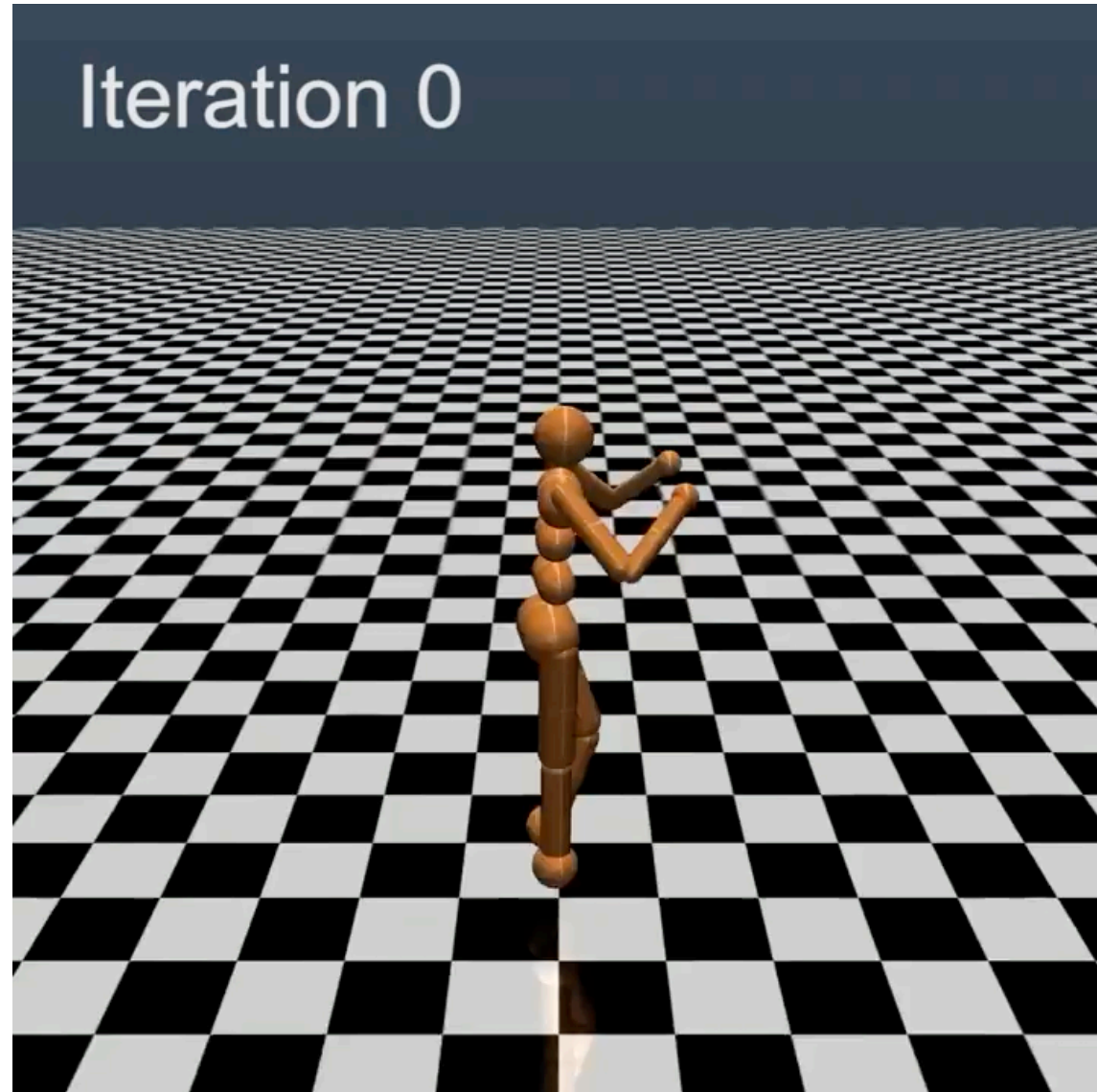
Humanoid



Half Cheetah

OpenAI Gym

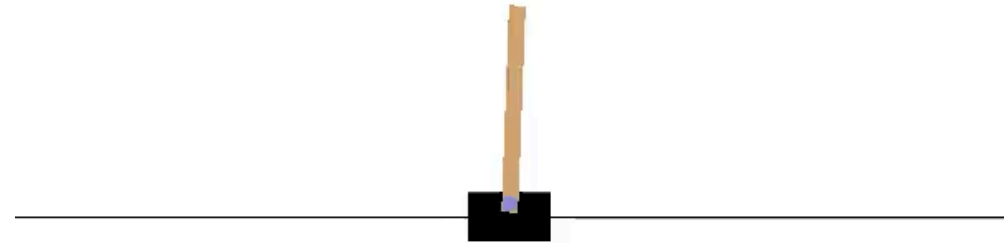
使用Gym库中的统一的标准数据集检测算法的优劣



Play CartPole Game

```
➡ import gym  
➡ env = gym.make( 'CartPole-v0' )
```

- Get the environment of CartPole from Gym.
- “env” provides states and reward.



Play CartPole Game

➡ `state = env.reset()`

➡ `for t in range(100):` render为渲染, 展示页面 A window pops up rendering CartPole.

➡ `env.render()`
`print(state)`

A random **action**.

➡ `action = env.action_space.sample()` 随机均匀的抽样一个动作Action, 实际上不用

➡ `state, reward, done, info = env.step(action)` Agent实际做这个动作

➡ `if done:` 是否结束 "done=1" means finished (win or lose the game)
`print('Finished')`
`break`

`env.close()`

Summary

Summary

Terminologies

- Agent 
- Environment
- State s
- Action a
- Reward r

Summary

Terminologies

- Agent 
- Environment
- State s
- Action a
- Reward r
- Policy $\pi(a|s)$
- State transition $p(s'|s, a)$

Summary

Terminologies

- Agent 
- Environment
- State s
- Action a
- Reward r
- Policy $\pi(a|s)$
- State transition $p(s'|s, a)$

Return and Value

- Return:

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

Summary

Terminologies

- Agent 
- Environment
- State s
- Action a
- Reward r
- Policy $\pi(a|s)$
- State transition $p(s'|s, a)$

Return and Value

- Return:

$$\underline{U}_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

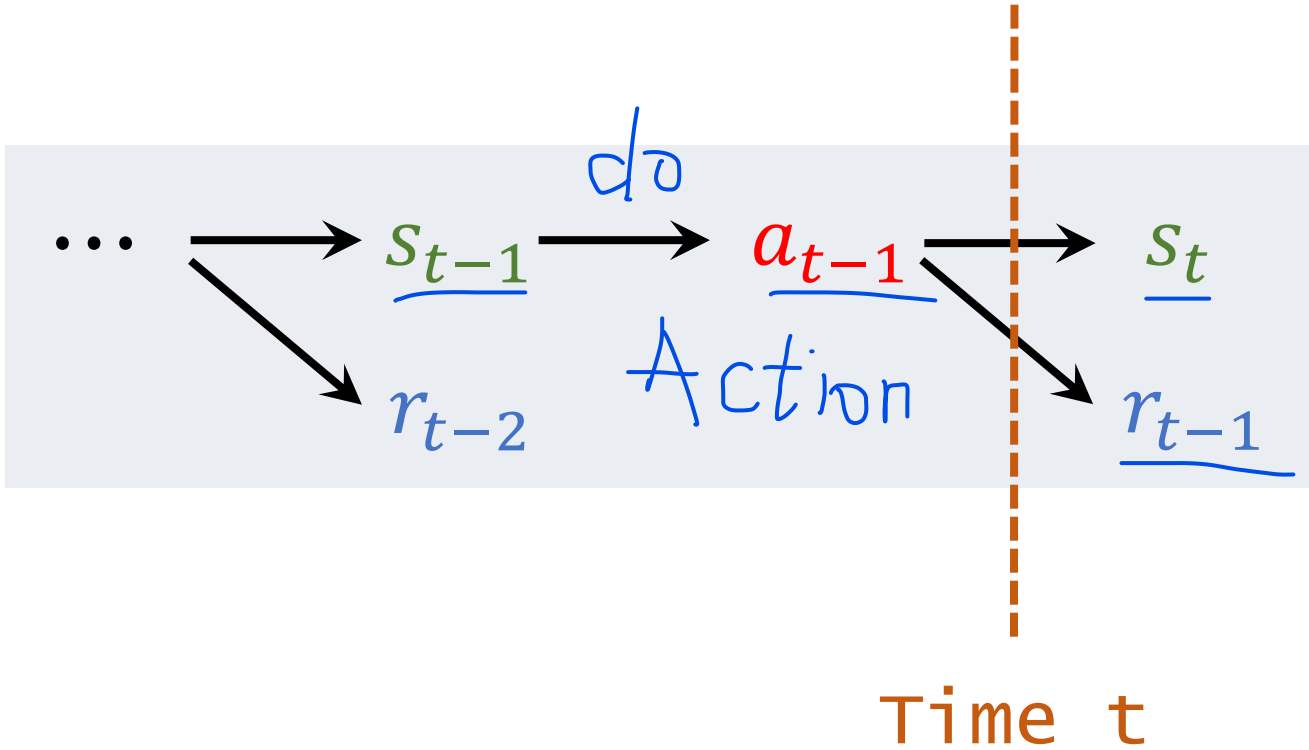
- Action-value function:

$$Q_\pi(s_t, a_t) = \mathbb{E} [\underline{U}_t | s_t, a_t].$$

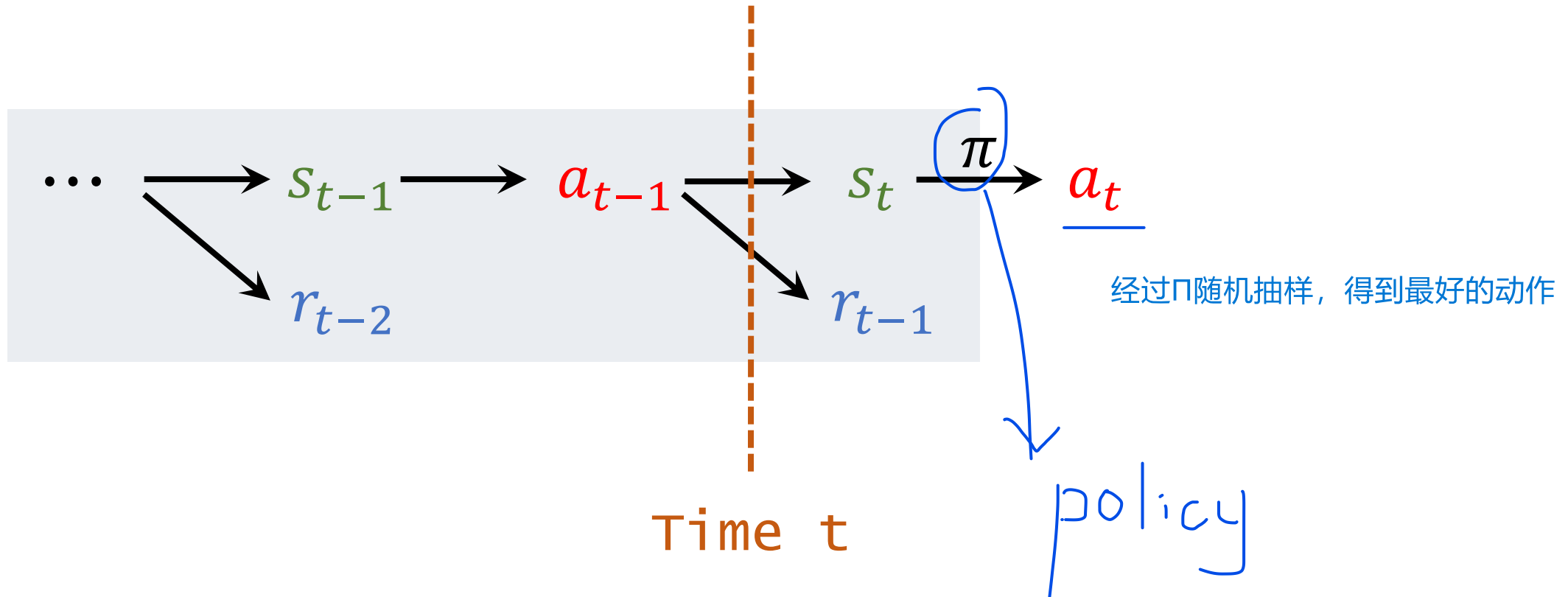
- State-value function:

$$V_\pi(s_t) = \mathbb{E}_{A \sim \pi} [Q_\pi(s_t, A)].$$

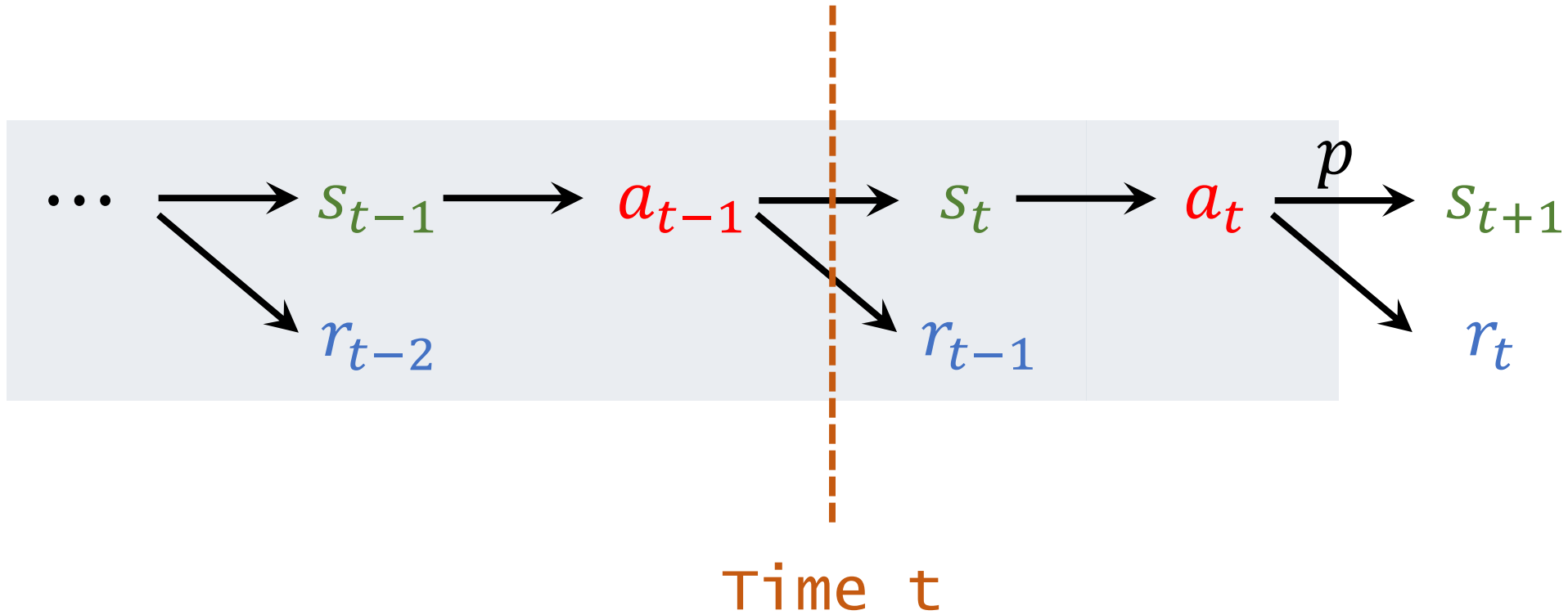
Play game using AI



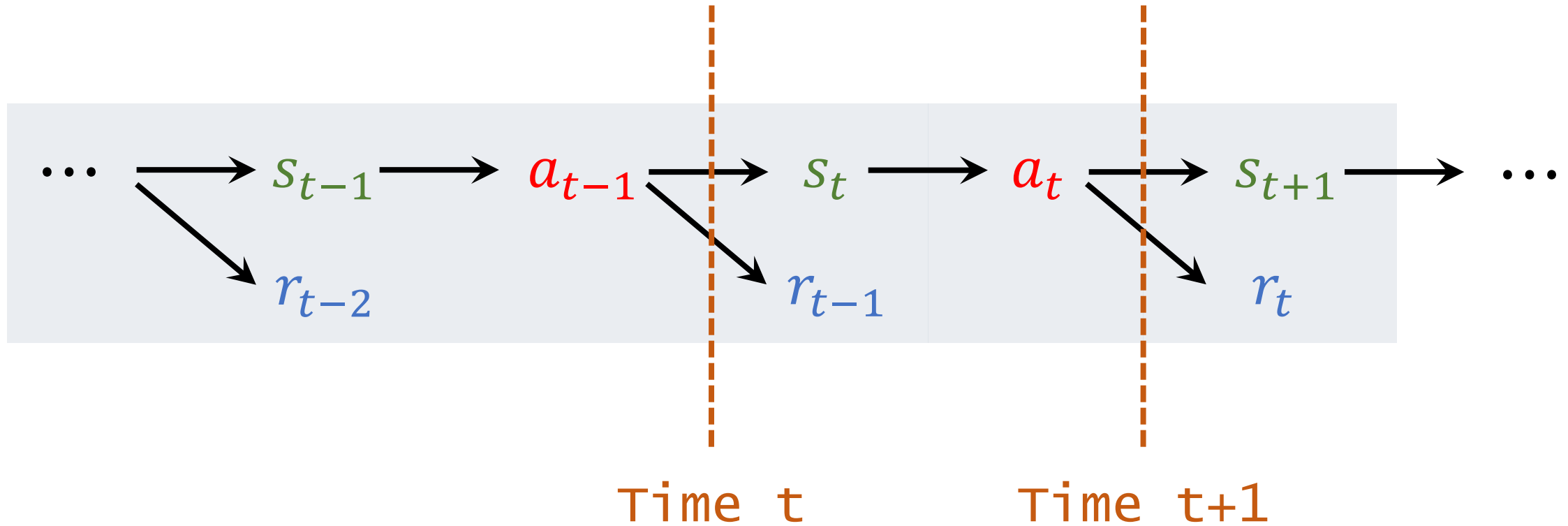
Play game using AI



Play game using AI



Play game using AI



Thank You!

<http://wangshusen.github.io/>